Hallazgos de observabilidad: orquestador multi-agente en Phoenix

Nov 10, 2026 · @Javier

Resumen

El orquestador pasa el 74 % de su tiempo esperando a la API de Gemini, alucinó en 1 de 7 consultas y una herramienta caída tumba al agente entero. Lo medí instrumentando el orquestador de la pre-entrega 6 con Arize Phoenix y OpenInference, sin tocar su código, y corriendo 7 consultas: 5 complejas y 2 fallas provocadas (proyecto `orquestador-m7-20261109-2002`).

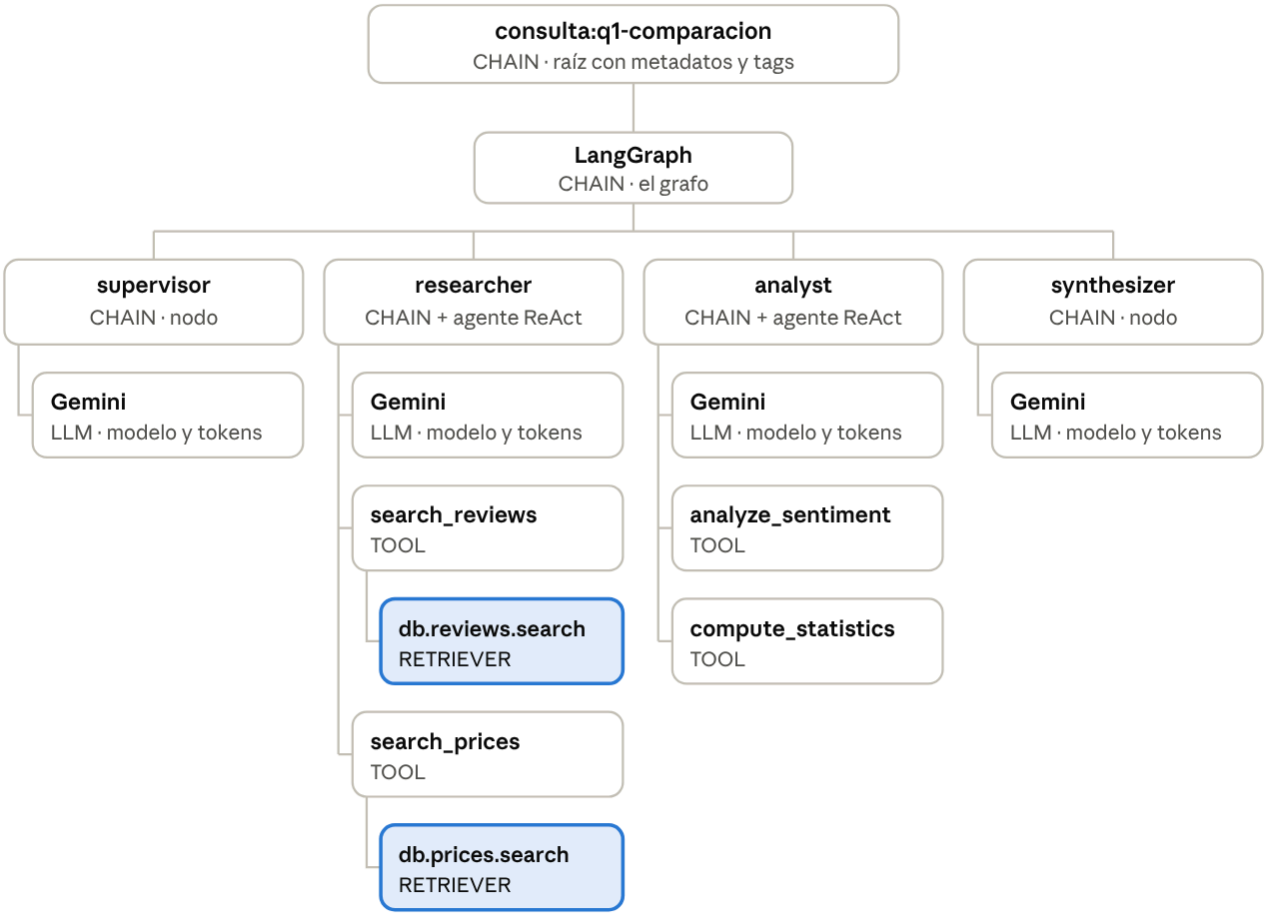
1. **Latencia:** el procesamiento local nunca pasa de 0,35 s por consulta. El p95 de 39,8 s lo infla una sola espera de 30 s después de un 429 (rate limit) en q2; sin esperas, el p95 es 21,4 s.
2. **Alucinación:** en f1 la respuesta dice "El iPhone 17 aún no ha sido lanzado al mercado", algo que ninguna herramienta devolvió. El chequeo numérico no lo detecta; el juez LLM sí.
3. **Errores:** en f2 la excepción de la fuente de reseñas subió 6 spans y cortó al investigador, que nunca pidió los precios, aunque esa fuente funcionaba.
4. **Costo:** el razonamiento más largo (q2) usó 12 llamadas al LLM y 11.123 tokens, unos USD 0,0073 en el plan pago. La primera y la última decisión del supervisor son predecibles y se pueden resolver sin LLM.

Jerarquía de spans

Cada consulta queda en una sola traza: agente → herramienta → modelo o fuente de datos. LangChainInstrumentor de OpenInference crea solo los spans del grafo, los agentes, las llamadas a Gemini (con modelo y tokens) y las herramientas.

Cada llamada al modelo y a la base cuelga del agente que la hizo

Árbol de spans de una consulta en Phoenix (q1-comparacion)



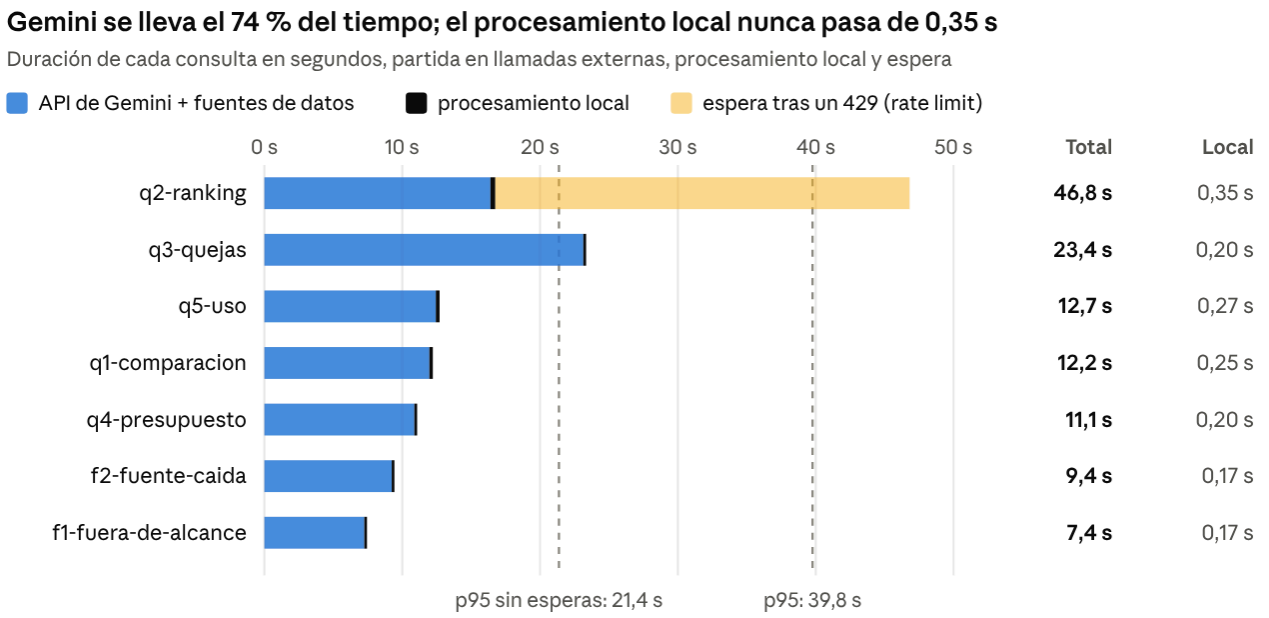
Resaltados: spans RETRIEVER que crea tracing.py, porque el instrumentador de LangChain no ve las fuentes de datos.

árbol de spans de q1-comparacion en Phoenix · 4 agentes

Las fuentes de datos no son objetos de LangChain, así que `tracing.py` las envuelve en spans `RETRIEVER` con la consulta y los documentos devueltos. Su padre se toma con `get_current_span()` de OpenInference; sin eso quedaban colgando de la raíz en vez de la herramienta que los llamó. Todos los spans llevan `session_id`, el tipo de consulta como tag, el escenario como metadata y la versión del prompt (`orquestador-v1.2`).

Latencia: red vs. procesamiento

El 74,2 % del tiempo se va en llamadas externas, el 24,4 % en un solo backoff y el procesamiento local va de 0,17 a 0,35 s por consulta. El p50 de las consultas es 12,2 s y el p95 39,8 s; cada llamada al LLM tarda 1,13 s en el p50 y 2,36 s en el p95.



results/trace_analysis.json · 7 consultas del 9/11/2026

Cómo lo medí: "externo" es la unión de los intervalos de los spans `LLM` y `RETRIEVER` (red más generación del modelo, que desde el cliente no se pueden separar). "Espera" son los huecos de más de 1 s sin ningún span trabajando. "Local" es el resto: cálculo de las herramientas y orquestación.

- **q2:** 30,1 de sus 46,8 s fueron la espera de `with_fallback` después de un 429 del analista. Esa sola espera lleva el p95 de 21,4 s a 39,8 s.
- **q3:** el span más lento de la sesión es una llamada del sintetizador de 12,6 s para solo 152 tokens de salida, cuando las demás llamadas tardan entre 1 y 3 s. Parece demora del lado de Google o un reintento interno del SDK.
- **Fuentes de datos:** entre 0 y 0,11 s, porque son simuladas en memoria.

Groundedness vs. alucinación

Seis respuestas son fieles a los datos de las herramientas; f1 alucinó y solo el juez LLM lo detectó. Usé dos evaluadores: un **chequeo numérico** en código (cada número de la respuesta tiene que aparecer en las salidas de las herramientas de esa traza o en la pregunta) y un **juez LLM** que compara cada afirmación con ese mismo contexto. Los dos veredictos quedaron como anotaciones en el span raíz de cada traza en Phoenix (`numeric_grounding` con `annotator_kind=CODE`, `faithfulness` con `annotator_kind=LLM`).

Consulta	Chequeo numérico	Juez LLM	Afirmación sin respaldo
f1-fuera-de-alcance	ok	**alucinación**	"El iPhone 17 aún no ha sido lanzado al mercado"
f2-fuente-caida	ok	fiel	—
q1-comparacion	ok	fiel	—
q2-ranking	ok	fiel	—
q3-quejas	ok	fiel	—
q4-presupuesto	ok	fiel	—
q5-uso	ok	fiel	—

Por qué f1 es una alucinación: las herramientas solo devolvieron que no había reseñas ni precios del iPhone 17. El sintetizador agregó un motivo sacado de su propio conocimiento, que además es falso. Lo correcto era decir que el producto no está en las fuentes. Como la frase no tiene números, el chequeo numérico la da por buena: por eso hace falta un juez LLM.

El chequeo numérico también se equivocó: en la primera versión marcó q1 como alucinación porque leía "$88.666,67" como 88,6667. Con el formato argentino de miles corregido, q1 pasó a fiel. Es el Error 3 de la unidad: ningún evaluador automático es la verdad.

Límite del juez: `gemini-3.8-flash` respondió 503 (alta demanda) en las 7 consultas, así que juzgó el modelo de respaldo, `gemini-flash-lite-latest`. Es el mismo modelo que escribió las respuestas, con riesgo de que se favorezca a sí mismo; aun así marcó su propia alucinación. Conviene repetir la evaluación con un juez de otra familia.

Errores y propagación

En f2 la caída de una sola fuente dejó al usuario sin reseñas y sin precios: el error nació en la base de reseñas y subió 6 spans, hasta cortar al agente investigador. Para provocarlo, `db.reviews.search` lanza `ConnectionError` dentro de su span `RETRIEVER`, así que la traza muestra la falla donde ocurre.

1. `db.reviews.search` (RETRIEVER) termina con estado ERROR: "fuente de reseñas no disponible".
2. La herramienta `search_reviews` no atrapa la excepción y también termina en ERROR.

3. El error corta el agente ReAct del investigador antes de que llame a `search_prices`, y el nodo `researcher` registra la contribución con error.
4. El supervisor reintenta con el investigador (flujo researcher → researcher → synthesizer), falla igual y pasa al sintetizador.
5. El sintetizador responde "No fue posible obtener las reseñas ni los precios". Es una respuesta fiel, pero los precios estaban disponibles: la fuente de precios nunca se consultó.

El problema es de diseño, no de la fuente: una herramienta que deja escapar una excepción tumba al agente completo. Si `search_reviews` devolviera `{"found": false, "error": "..."}`, el modelo seguiría con `search_prices` y la respuesta tendría al menos los precios. El reintento del supervisor tampoco sirve frente a una caída: vuelve a llamar al LLM y falla igual.

En q2 se vio otro error: un 429 (rate limit) del analista subió 4 spans, pero el reintento de `with_fallback` lo absorbió y la respuesta salió completa, 30 s más tarde.

Tokens y costo del razonamiento más largo

La consulta q2 (ranking de 3 productos) fue la más larga: 12 llamadas al LLM, 9.306 tokens de entrada y 1.817 de salida, unos USD 0,0073. Los tokens de salida son el 16 % del total pero el 62 % del costo, porque cuestan 8 veces más que los de entrada.

Agente	Llamadas	Tokens de entrada	Tokens de salida	Costo est. (USD)
analyst	4	1.804	1.036	0,0031
supervisor	4	2.667	335	0,0016
researcher	3	3.172	194	0,0014
synthesizer	1	1.663	252	0,0011

Costo estimado con precios del plan pago para `gemini-flash-lite-latest`, valuado como `gemini-3.5-flash-lite` (USD 0,30 por millón de tokens de entrada y USD 2,50 por millón de salida, según morphllm.com, al 25/10/2026). El plan gratuito no cobra. Toda la sesión, 7 consultas, sumó 40.920 tokens de entrada y 7.675 de salida, unos USD 0,031.

Observación de eficiencia: el analista es el agente más caro, con el 43 % del costo de q2, y casi todo es salida: escribe un análisis en markdown de 1.036 tokens que después el sintetizador reescribe en 252. Si el analista devolviera solo los números estructurados (sentimiento, promedios, ranking), con un tope de unos 300 tokens, q2 costaría un 25 % menos (unos USD 0,0018 por consulta). Hay dos ahorros más:

- La primera decisión del supervisor siempre es researcher y la última siempre es FINISH. Resolverlas en código ahorra 2 de las 12 llamadas.
- El investigador manda al LLM el texto completo de las reseñas en cada vuelta: es el agente con más tokens de entrada (3.172). Con `response_format="content_and_artifact"` el modelo vería un resumen y los datos completos viajarían por el estado.

Recomendaciones y limitaciones

La mejora más urgente es que las herramientas atrapen sus errores: es la única falla que le quitó información al usuario.

Prioridad	Cambio	Qué resuelve	Evidencia en las trazas
1	`search_reviews` y `search_prices` devuelven un dict de error en vez de lanzar la excepción	Una fuente caída ya no tumba al investigador	f2: el error subió 6 spans y nunca se pidieron los precios
2	Prompt del sintetizador: si no hay datos, decirlo sin dar motivos que no vengan de las herramientas	Alucinaciones con conocimiento propio del modelo	f1: "aún no ha sido lanzado al mercado"
3	Salida estructurada y corta del analista (unos 300 tokens)	Costo: el analista es el 43 % de q2	q2: 1.036 tokens de salida del analista
4	Primera y última decisión del supervisor en código	2 llamadas menos por consulta	Las 7 trazas empiezan en researcher y terminan en FINISH
5	Un span propio para la espera del backoff	La espera aparece en Phoenix en vez de deducirse de un hueco	q2: 30 s sin spans
6	Correr el juez LLM en cada despliegue, con un modelo de otra familia	Detectar regresiones de fidelidad	f1 solo se detectó con el juez

Limitaciones de este análisis:

- Son 7 consultas: el p50 y el p95 dan una idea, no una medición estable.

- La latencia "externa" junta red y generación del modelo: desde el cliente no se pueden separar. El caso q3 (un span de 12,6 s para 152 tokens de salida) parece demora del proveedor, pero no lo puedo confirmar desde la traza.
- Las fuentes de datos son simuladas, en memoria: sus 0,05 s no representan una base real.
- Los costos son una estimación con precios públicos de terceros; el plan gratuito no cobra.